OpenMP CSR SpMV 实验报告

1. 策略、环境与正确性

CSR 用 row_ptr 定位每行在 values 和 col_idx 中的范围；每行独立计算一个 y[row]。三个策略均只读矩阵与 x，每行由一个线程写回，累加变量在线程内独立，无需原子加法。

- **策略 1：逐行 guided**。外层行循环使用 `schedule(guided)`，行内顺序累加。
- **策略 2: guided,16 + SIMD**。使用 `schedule(guided, 16)`，行内使用 `omp simd reduction(+:sum)`。
- **策略 3：64 行分块 guided**。先将连续 64 行作为一个循环迭代，再使用 guided 分配这些块；块内和行内顺序执行。guided 初期可能一次领取多个块，最后一块处理不足 64 行的余数。

项目	配置
CPU 与系统	Apple M4，10 个物理核（4P + 6E），macOS 26.7.1
编译器与运行时	Homebrew Clang 23.1.2、LLVM libomp，OpenMP 202011
编译选项	`-O2 -std=c++17 -fopenmp -pthread`；采样版额外加 `-g`
默认矩阵	10000×100000，958976 个非零存储条目
行长与随机性	普通行各 32 个条目；32 个长行各 20000 个；种子 20260926
主基准配置	8 线程，`OMP_DYNAMIC=FALSE`，未显式绑定 P/E 核

同组实现使用相同输入、编译选项和计时范围。主基准独立运行 10 轮，每轮轮换策略顺序；每个实现先预热 1 次、全量校验，再用 steady_clock 正式调用 5000 次，报告单次平均耗时的中位数及范围。生成、预热、校验和输出均不计时。

保留串行参考，所有 y[row] 逐元素比较，绝对和相对误差容差均为 `1e-10`。SIMD 改变累加顺序，允许容差内的舍入差异，未放宽检查。最终源码在默认矩阵 1、2、4、8、10 线程及空行、单长行、65 行分块尾部共 8 组输入上均通过检查，真实终端截图见附录 A；主基准 50 次进程运行也全部通过。

本报告对应源码 SHA256 前缀为 **35c07ac4388e6229**。早期策略 3 的 dynamic 版本仅保留在探索记录中，不作为当前策略 3 的成绩。

2. 运行时间、加速比与并行效率

T1 定义为本组串行中位耗时，S=T1/Tp，E=S/p；各组单独测串行基线，不混用不同环境或批次的时间。下表为默认矩阵、8 线程、10 轮 ×5000 次的最终版本结果。

实现	中位耗时/ms	最小–最大/ms	加速比	效率
串行	0.748359	0.731852–0.757485	1.000×	—
std::thread	0.246725	0.239328–0.255293	3.033×	37.91%
逐行 guided	0.189346	0.170881–0.210680	3.952×	49.40%
guided,16 + SIMD	0.162950	0.146843–0.174546	4.593×	57.41%
64 行块 guided	0.192445	0.185624–0.207118	3.889×	48.61%

本次策略 2 最快，相对串行加速 **4.593 倍**。逐行 guided 比 std::thread 耗时低约 **23.3%**。64 行块 guided 未优于逐行 guided，说明本矩阵上增加块粒度没有带来净收益；这里不能单凭总时间区分调度成本与负载不均。

3. std::thread 对比与 SIMD 消融

两者都适合按行独立计算 SpMV。std::thread 基线按连续行分配，每次调用创建并 join 全部工作线程；短任务反复承担这一成本，且按行均分不保证非零元素数均衡。OpenMP 将循环分配、线程数和调度表达为指令，运行时通常复用线程，便于快速比较方案。

std::thread 适合需要自定义线程生命周期、任务队列、通信和专用工作线程的场景；其优势是控制直接。线程池可能改善本基线，但本实验未实现或测试，不能据此认定 OpenMP 总是更快。

为单独判断 SIMD 的作用，固定 8 线程与 **guided,16**，保持行内算式、输入和编译选项相同，只关闭或开启 SIMD。独立测量 10 轮 ×5000 次，轮换两者顺序，每次均先预热并校验。

行内实现	中位耗时/ms	最小–最大/ms
顺序累加，关闭 SIMD	0.184491	0.171050–0.206685
SIMD 归约	0.161849	0.144538–0.169493

开启 SIMD 后中位耗时降低 **12.27%**，支持保留行内 SIMD。该消融与上一表分开测量，不将跨批次时间差解释为某项开销。直接比较策略 1 与策略 2 还同时改变最小 chunk，因此不把该比较称为纯 SIMD 消融。

4. 参数实验与重复性

研究线程数、矩阵规模、调度 chunk 三类变量。前两类引用先前 guided 策略 1 的 10 轮 ×500 次记录，已核对其矩阵生成和 guided 内核与当前源码相同；不引用旧策略 2、3 的结果代表当前版本。

线程数与矩阵规模

线程数实验固定默认矩阵；规模实验固定 8 线程、100000 列，行数与长行数同比变化，长行比例保持 0.32%。各配置使用自己的串行基线。

线程数	guided/ms	加速比	效率
1	0.732214	0.993×	99.29%
2	0.385775	1.896×	94.80%
4	0.207377	3.515×	87.87%
8	0.166548	4.417×	55.22%
10	0.166808	4.262×	42.62%

行数	非零条目数	guided/ms	加速比
2500	239744	0.067470	2.675×
10000	958976	0.166548	4.417×
40000	3835904	0.551362	5.318×

1→8 线程时收益明显，8→10 线程基本持平；增加线程并非线性提速。规模增大时加速比提高，与固定开销被摊薄相符，但本实验未单独量化各项成本。

guided 最小 chunk 与顺序复测

固定 4 线程、默认矩阵，10 轮 ×5000 次。每轮随机排列五个 chunk，最后测串行。两次完整实验沿用相同排列种子 20261017，以观察相同顺序下的重复性。chunk 是最小块行数，不是所有任务固定的行数；最后一块可更小。

chunk	随机第 1 次/ms	随机第 2 次/ms	第 2 次加速比
1	0.210475	0.217074	3.424×
16	0.213844	0.215183	3.454×
32	0.213050	0.219627	3.384×
64	0.216615	0.224274	3.314×
128	0.227619	0.227751	3.263×

两次串行基线分别为 0.740799、0.743180 ms。第一次 chunk=1 最快，第二次为 16，1/16/32 的小差距与跨轮波动并存；128 在两次随机实验中均较慢。固定倒序中 128 曾最快，复测未保持该排序。当前保留默认 chunk=1，不将单次排名认定为普遍最优；七种 chunk 的完整实验及顺序记录见附带 CSV。

5. Instruments 性能分析

按本机 macOS 环境使用 Instruments 27.0 的 Time Profiler 和 CPU Counters。此项替代了作业 3.5 指定的 Linux perf，工具形式与原要求不同，未将两者说成相同工具。

对最终源码的串行、策略 1、std::thread 分别录制一次 Time Profiler，8 线程、10000 次调用，先预热校验再计时。采样覆盖整个进程，包含准备阶段；下表为栈顶 Self 权重占比，不是墙钟时间或精确开销分解。正常耗时仍采用第 2 节未开采样器的重复实验。

实现	SpMV 核心 Self	观察到的运行时热点
串行	99.64%	热点主要在 SpMV 循环
guided	95.25%	任务分配函数合计约 1.01%，条件变量等待/通知约 2.66%
std::thread	90.62%	创建线程相关函数合计约 2.29%，另有等待及清理

std::thread 的 trace 中采样到 9315 个不同线程 ID，guided 为 8 个，支持反复创建回收线程的解释；9315 不是同时运行的线程数。创建函数的 Self 占比也不是全部线程管理成本，不能将其直接从墙钟时间中扣除。

补充 CPU Counters 使用独立的逐行 static、guided、dynamic,1 对照内核，均为 4 线程、3 次 trace×10000 次调用，仅统计预热后 MeasuredSpMV 区间。它们不是当前三个提交策略；下列 IPC 来自计数器采样，L1D 为推测执行 Load Miss/Load µop。

对照调度	采样 IPC	L1D Load Miss/Load µop
static	1.989	32.14%
guided	1.986	31.51%
dynamic,1	2.153	29.20%

较高 IPC 或较低 L1D 比值不保证任务更快，运行库的额外指令与访存也会改变分母。未采集 L2 未命中或实际 DRAM 流量，不据此断言存在缓存或内存带宽瓶颈。

6. 结论、局限性与来源

在本次输入上保留 guided,16 + SIMD；固定调度参数的消融支持 SIMD 有收益。按行并行实现简单且正确，分块与调度参数需要通过重复实验判断，不能仅凭一次最快结果决定。

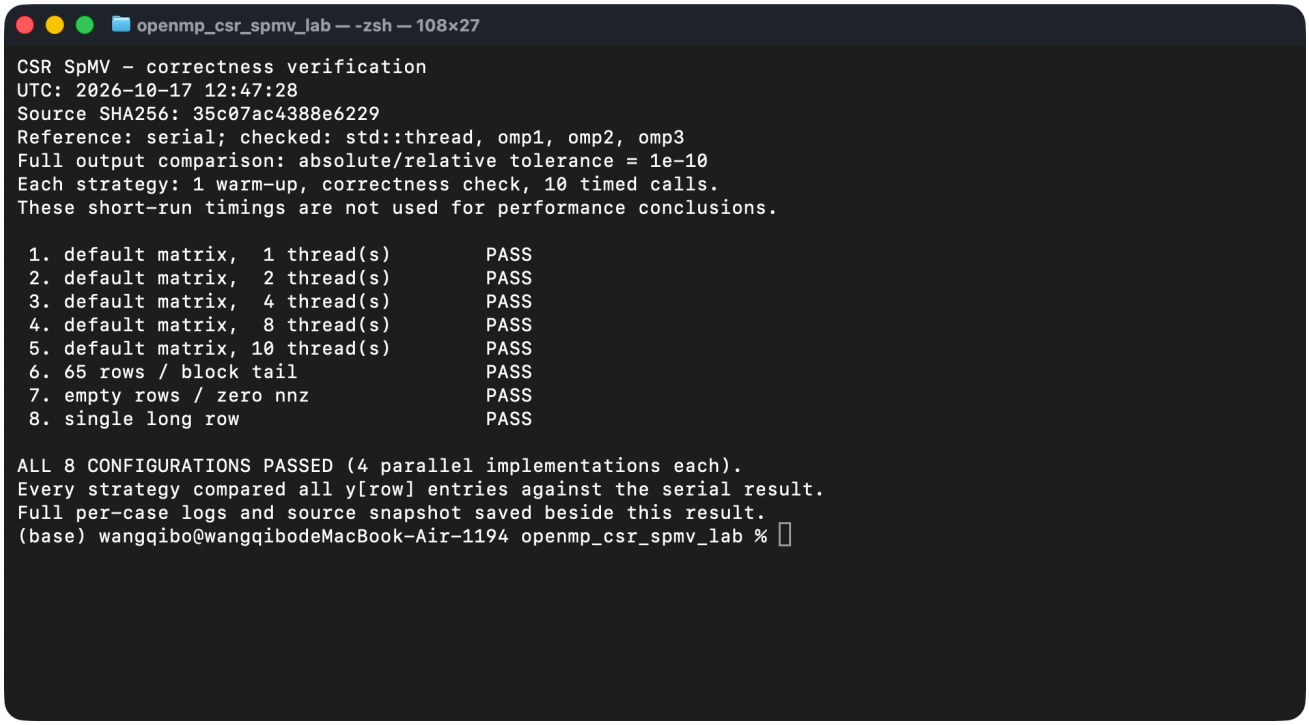
结论限于该矩阵族及本机。未固定 P/E 核、频率和温度，跨轮存在波动；8→10 线程收益有限，不代表其他机器也如此。未来可用多种偏斜分布、固定线程位置或 std::thread 线程池进一步检验，当前不宣称这些方案已经验证。

AI 使用声明：使用 Codex 辅助解释代码、编写测试与分析工具、整理报告；所列数值来自实际运行及 trace，并核对原始记录。报告结论需由本人复核。个人心路历程另行由本人撰写，不由 AI 代写或润色。

参考资料：OpenMP 5.2 规范、SIMD reduction、Apple Instruments CPU 性能分析。

附录 A：正确性结果真实截图

以下截图来自实际运行验证工具的 macOS Terminal 窗口，非重绘日志。8 个配置均对完整输出逐元素校验；每个配置包含 std::thread 和三个 OpenMP 实现，使用同一串行参考。



真实终端正确性检查：8 个配置全部通过

对应源码 SHA256：35c07ac4388e62290b87f4e032dca66c653a27678e28d8d7688f7cc0b5333440。

完整日志位于截图目录的 logs/，含 results.json 和 case_1.txt 至 case_8.txt。这些短运行仅用于正确性，不用于性能结论。

附带数据：主基准 benchmark_three/、消融 simd_ablation/、Time Profiler instruments/；线程数与规模见 output/parameter_results/，chunk 两次随机复测见对应目录。历史完整探索报告已移至 output/final_report/report_before_completion.md，不作为当前版本结论的依据。